



**TECNOLÓGICO
NACIONAL DE MÉXICO**



TECNM - INSTITUTO TECNOLÓGICO DE CULIACÁN

ASIGNATURA:

RESIDENCIAS PROFESIONALES

CARRERA: INGENIERÍA EN SISTEMAS COMPUTACIONALES

TRABAJO:

**METODOLOGÍA PARA LA DETECCIÓN DE TIPO DE CULTIVO MEDIANTE
TÉCNICAS DE IA**

TÍTULO:

**REPORTE DE ACTIVIDAD 1: ANALIZAR, IMPLEMENTAR Y VALIDAR EL
COMPONENTE DE PLANIFICACIÓN DE VUELO**

NOMBRE DEL MAESTRO:

ZURIEL DATHAN MORA FÉLIX

NOMBRE DEL ESTUDIANTE(S):

RODRÍGUEZ MORENO AMÍLCAR

CULIACÁN SINALOA, 13 DE SEPTIEMBRE DEL 2026.

1. Índice

1. Índice	1
2. Introducción	2
3. Objetivos de la investigación	2
3.1 Objetivo general	2
3.2 Objetivos específicos	3
4. Justificación	3
5. Alcance del proyecto	4
5.1 Qué incluye	4
5.2 Qué no incluye	4
6. Desarrollo	4
6.1 Análisis del problema logístico	4
6.3 Justificación del algoritmo heurístico	6
6.4 Implementación computacional	9
6.5 Evaluación del rendimiento de la solución	16
8. Conclusiones	21

2. Introducción

El presente proyecto aborda el problema de optimizar la ubicación de sensores de humedad en campos agrícolas de la región de Guasave, Sinaloa, con el propósito de mejorar la estimación de las condiciones del suelo y apoyar una gestión más eficiente del riego. Cada sensor representa un punto geográfico dentro del terreno en el que se estiman variables ambientales como humedad, temperatura, salinidad y tipo de cultivo a partir de un conjunto de 100 registros georreferenciados. La combinación de estas variables permite evaluar la calidad de una configuración de sensores mediante una función fitness, cuyo objetivo es maximizar la capacidad del sistema para representar de manera precisa las condiciones agrícolas del área de estudio.

Debido a que cada solución está formada por múltiples sensores y cada uno de ellos puede ubicarse en cualquier coordenada dentro de los límites del terreno, el número de configuraciones posibles crece rápidamente conforme aumenta la cantidad de sensores considerados. Esta característica convierte al problema en uno de optimización continua de alta dimensionalidad, en el que resulta poco práctico utilizar métodos exactos de búsqueda. En este contexto, el algoritmo de Optimización por Enjambre de Partículas (Particle Swarm Optimization, PSO) constituye una alternativa adecuada, ya que permite explorar el espacio de soluciones mediante un proceso iterativo inspirado en el comportamiento colectivo de sistemas naturales. En este proyecto, el algoritmo PSO se utiliza para encontrar configuraciones de sensores que maximicen el valor de la función fitness y, en consecuencia, mejoren la precisión en la estimación de la humedad y el aprovechamiento de los recursos hídricos en el sistema agrícola.

Los informes y documentación de los avances se publicarán en el siguiente repositorio:

https://github.com/ramilcar525/Automatic-FlightPlan_Documentacion

3. Objetivos de la investigación

3.1 Objetivo general

Optimizar la ubicación de sensores de humedad en campos agrícolas de la región de Guasave, Sinaloa, mediante la implementación del algoritmo de Enjambre de Partículas (PSO), con el fin de mejorar la eficiencia del riego y el aprovechamiento de los recursos hídricos.

Desarrollar e implementar un modelo de optimización basado en el algoritmo de Enjambre de Partículas (PSO) que permita determinar la distribución más adecuada de sensores de humedad en terrenos agrícolas de Guasave, Sinaloa, con el propósito de obtener estimaciones más precisas de las condiciones del suelo y contribuir a un uso más eficiente del agua de riego.

3.2 Objetivos específicos

- Modelar el problema de ubicación de sensores como un espacio de búsqueda multidimensional.
- Implementar el algoritmo PSO para encontrar configuraciones óptimas de sensores considerando topografía, cultivos y suelo.
- Evaluar el desempeño del algoritmo mediante simulaciones y análisis de eficiencia hídrica.

4. Justificación

La ubicación estratégica de sensores de humedad constituye un problema fundamental en los sistemas modernos de agricultura de precisión, ya que influye directamente en la calidad de la información utilizada para la toma de decisiones sobre riego, fertilización y manejo del suelo. Una distribución adecuada de sensores permite estimar con mayor exactitud las variaciones espaciales de humedad, temperatura y salinidad, lo que se traduce en un mejor aprovechamiento de los recursos hídricos y en una reducción del desperdicio de agua. Sin embargo, determinar manualmente las posiciones óptimas resulta complejo debido a la interacción simultánea de múltiples variables ambientales y a la gran cantidad de configuraciones posibles.

En este contexto, los métodos heurísticos representan una alternativa eficiente para abordar el problema sin recurrir a procedimientos exhaustivos de alto costo computacional. La implementación del algoritmo de Optimización por Enjambre de Partículas (PSO) permite explorar de manera iterativa distintas configuraciones de sensores, aprovechando tanto la experiencia individual de cada solución como el conocimiento colectivo del enjambre. Este enfoque ofrece un modelo flexible y escalable, capaz de adaptarse a diferentes tamaños de terreno, cantidades de sensores y criterios de evaluación. Además de generar soluciones de alta calidad, el proyecto proporciona una base metodológica sólida para incorporar en el futuro otras técnicas de optimización y análisis orientadas al diseño de sistemas agrícolas más eficientes y sostenibles.

5. Alcance del proyecto

5.1 Qué incluye

- Modelado del problema de ubicación de sensores como un problema de optimización continua en dos dimensiones.
- Implementación del algoritmo de Optimización por Enjambre de Partículas (PSO).
- Diseño de una función fitness basada en humedad, temperatura, salinidad y tipo de cultivo.
- Generación y evaluación de configuraciones de sensores dentro de un terreno agrícola georreferenciado.
- Análisis de resultados mediante tablas de indicadores y gráficas de convergencia del algoritmo.

5.2 Qué no incluye

- Medición explícita de redundancia o superposición entre sensores.
- Implementación de restricciones físicas de instalación o costos económicos de adquisición.
- Validación del modelo con mediciones reales en campo.

6. Desarrollo

6.1 Análisis del problema logístico

Se busca optimizar la ubicación de sensores de humedad en campos agrícolas de la región de Guasave, Sinaloa, mediante la implementación del algoritmo de Enjambre de Partículas (PSO), con el fin de mejorar la eficiencia del riego y el aprovechamiento de los recursos hídricos.

Se establece un Data set de 100 puntos con las variables de Humedad (%), Cultivo, Elevación (msnm), Salinidad (dS/m), Temperatura (°C), Latitud y Longitud; y se buscan establecer:

- Una representación de solución del problema en una partícula perteneciente a un enjambre que satisfaga el modelo del algoritmo PSO
- Una función fitness que permita evaluar cada partícula tomando en cuenta las variables del Data set
- Un método de representación de las fuerzas que permitan evolucionar a cada partícula como evolución del enjambre

Cada punto del Data set representan zonas de cultivo, las variables que se consideran en estos puntos son:

- Humedad: Representa el porcentaje de humedad ya medida en el aire dentro de la zona de cultivo, finalmente es lo que se busca medir con los sensores por lo que es la variable más fundamental para el algoritmo de búsqueda.
- Cultivo: Representa qué es lo que se cultiva en esa área, es una variable discreta de 3 posibles estados. Su efecto dentro del algoritmo es establecer pesos de prioridad en la humedad de los puntos que influyen en el cálculo de la función fitness.
- Elevación: Representa los metros sobre el nivel del mar en los que se encuentra la zona de cultivo. No tiene una representación como dimensión al representar una partícula, pero afecta en el cálculo de la distancia de coordenadas entre los puntos del mapa, finalmente siendo algo con un leve efecto en el cálculo del peso de los valores de humedad que afectan a la función fitness.
- Salinidad: Mide en decisiemens por metro la conductividad eléctrica del suelo, dato clave para medir su salinidad. La salinidad afecta la capacidad del suelo para retener agua y distribuir humedad de forma uniforme, por lo que influye indirectamente en la precisión de estimación de humedad entre puntos cercanos. Dentro del algoritmo, esta variable funciona como un factor correctivo en el cálculo del peso de influencia de los puntos vecinos, reduciendo o aumentando la influencia de ciertos valores de humedad dependiendo de las diferencias de salinidad entre zonas.
- Temperatura: Representada en grados Celsius, influye directamente en la evaporación y pérdida de humedad del suelo. Zonas con temperaturas más elevadas presentan una variación de humedad más rápida, por lo que esta variable se utiliza como un factor de sensibilidad dentro de la función fitness. Los errores de estimación en puntos con temperaturas extremas pueden recibir una ponderación mayor, debido a que representan regiones más críticas para el control eficiente del riego.
- Latitud y Longitud: Los valores de las coordenadas que representan tanto la ubicación de las zonas de cultivo, como la ubicación de los sensores simulados. Son las dimensiones de las partículas, tienen la función tanto de representar a los sensores, como de ser las variables principales en el cálculo de distancia, factor principal en el cálculo del peso de una humedad para la función fitness.

El objetivo principal del sistema es determinar la ubicación óptima de un conjunto de sensores capaces de estimar con la mayor precisión posible la humedad presente en distintas zonas de cultivo. Aunque la humedad es la variable de interés final, su distribución espacial no es uniforme y depende de diversos factores ambientales y agronómicos. Por esta razón, la decisión sobre dónde colocar cada sensor no se basa únicamente en la proximidad geográfica entre puntos, sino también en las características particulares de los cultivos cercanos. Cada sensor representa una posición hipotética dentro del terreno y, a partir de esa ubicación, estima la humedad considerando la influencia de todos los puntos del Data set, otorgando mayor importancia a aquellos que se encuentran más próximos y cuyas condiciones son más relevantes para el análisis. De esta manera, el algoritmo busca posiciones que no solo representen adecuadamente la distribución espacial del terreno, sino que también se concentren en las zonas donde la medición de humedad resulta más crítica para optimizar la gestión del riego.

6.3 Justificación del algoritmo heurístico

Debido a que el número de combinaciones posibles crece exponencialmente con la cantidad de sensores y a que la función objetivo depende de múltiples variables ambientales y agronómicas, el problema presenta una alta complejidad computacional y no resulta práctico resolverlo mediante métodos exhaustivos. Por esta razón se seleccionó el algoritmo de Optimización por Enjambre de Partículas (Particle Swarm Optimization, PSO), una metaheurística inspirada en el comportamiento colectivo de organismos, en donde cada individuo explora el espacio de soluciones y ajusta su trayectoria de acuerdo con su propia experiencia y la experiencia compartida por el grupo.

El algoritmo desarrollado puede dividirse en cinco fases generales: parametrización del modelo, generación del enjambre inicial, evaluación de partículas, evolución del enjambre y análisis de resultados. Estas fases permiten estructurar el proceso de búsqueda de manera ordenada, desde la definición de los criterios de evaluación hasta la obtención de una configuración final de sensores capaz de maximizar la calidad de estimación de humedad en la región de estudio.

Parametrización del modelo

Consiste en establecer todos los parámetros necesarios para el funcionamiento del algoritmo. Se definen, por una parte, los pesos utilizados en la función fitness,

los cuales determinan la importancia relativa de variables como la temperatura, la salinidad y el tipo de cultivo. Asimismo, se asignan ponderaciones específicas a los cultivos considerados en el estudio, tales como maíz, chile y tomate, permitiendo que ciertas zonas agrícolas influyan con mayor intensidad en el proceso de optimización. Por otra parte, se configuran los parámetros propios del algoritmo PSO: el número de partículas del enjambre, el número de iteraciones, el factor de inercia w , y los coeficientes cognitivo y social c_1 y c_2 . Estos parámetros regulan el equilibrio entre exploración y explotación, es decir, entre la búsqueda de nuevas regiones del espacio y el refinamiento de las mejores soluciones encontradas.

Generación del enjambre inicial

Crear el conjunto inicial de soluciones candidatas. Cada partícula representa una posible distribución de sensores y está compuesta por tres elementos: la posición actual, el vector de velocidades y el óptimo local. La posición actual contiene las coordenadas geográficas de todos los sensores y un valor fitness asociado; el vector de velocidades indica la magnitud y dirección del desplazamiento de cada coordenada; y el óptimo local almacena la mejor configuración encontrada hasta ese momento por la partícula.

Las coordenadas iniciales se generan aleatoriamente dentro de los límites geográficos del conjunto de datos, utilizando los valores mínimos y máximos de latitud y longitud registrados en el Data set. De manera similar, las velocidades se inicializan con valores pequeños aleatorios para permitir un desplazamiento gradual. Esta estrategia garantiza una distribución inicial diversa de partículas, favoreciendo una exploración amplia del espacio de búsqueda desde el comienzo del algoritmo.

Evaluación de partículas

La tercera fase corresponde a la evaluación de cada partícula mediante la función fitness. Esta función estima, para cada coordenada propuesta, los valores ambientales asociados utilizando los puntos más cercanos del Data set y asignando pesos en función de la distancia y de variables correctivas como la salinidad, la temperatura y el tipo de cultivo. El resultado es una estimación de la humedad y de otros indicadores relevantes para cada sensor simulado, luego se calcula un valor fitness individual por sensor y se obtiene la suma de todos ellos como fitness total de la partícula. Este valor representa la calidad global de la configuración propuesta y constituye el criterio principal para comparar soluciones.

Durante esta fase también se actualiza el óptimo local de cada partícula, reemplazándolo cuando la nueva posición supera el rendimiento previamente alcanzado, y se identifica la mejor partícula del enjambre en la iteración actual.

Evolución del enjambre

La cuarta fase corresponde al mecanismo de búsqueda iterativa del algoritmo. En cada iteración, todas las partículas actualizan sus posiciones utilizando la ecuación clásica del PSO:

$$v_i(t + 1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (p_i^{best} - p_i(t)) + c_2 \cdot r_2 \cdot (p_{gbest} - p_i(t))$$

$$x_{t+1} = x_t + v_{t+1}$$

En esta expresión, el término de inercia conserva parte del movimiento previo, el término cognitivo dirige a la partícula hacia la mejor solución que ha encontrado individualmente, y el término social la atrae hacia el mejor resultado compartido por el enjambre. Los valores aleatorios r_1 y r_2 introducen variabilidad y evitan trayectorias deterministas, favoreciendo la exploración del espacio de soluciones.

Antes de aplicar esta ecuación, es necesario establecer una correspondencia consistente entre los sensores de la partícula y los del óptimo social. Para ello se implementa un procedimiento de ordenamiento basado en distancias euclidianas, el cual asigna cada sensor al punto del óptimo social más adecuado mediante un criterio de prioridad. De esta forma, las fuerzas de atracción actúan sobre sensores equivalentes y el movimiento conserva coherencia espacial a lo largo del proceso evolutivo.

Después de actualizar las posiciones y velocidades, el enjambre es evaluado nuevamente, se registra el mejor fitness obtenido en la iteración y se compara con el óptimo social. Como lo indica el PSO, si la nueva mejor partícula supera el rendimiento histórico del algoritmo, ésta se convierte en el nuevo óptimo social. Este proceso se repite hasta completar el número establecido de iteraciones.

Análisis y presentación de resultados

La fase final consiste en analizar la mejor solución encontrada. Se presenta la partícula ganadora con las coordenadas de sus sensores, los valores estimados de humedad, elevación, salinidad y temperatura para cada uno de ellos, así como sus fitness individuales. También se calculan los promedios globales de las

variables ambientales y la sumatoria total de la función fitness, permitiendo interpretar el comportamiento de la solución en términos agronómicos. Además, se calcula el porcentaje de mejora respecto a la mejor solución obtenida en la inicialización mediante la evaluación *fitness* del óptimo final, lo cual permite cuantificar la eficiencia del proceso heurístico. Finalmente, se genera una gráfica de líneas con el historial de fitness del óptimo del enjambre a lo largo de las iteraciones, proporcionando una representación visual de la convergencia del algoritmo y de la evolución de la calidad de las soluciones encontradas.

6.4 Implementación computacional

El algoritmo se desarrolló en Python y puede interpretarse de manera directa siguiendo las cinco fases descritas en la justificación del algoritmo heurístico. Cada fase corresponde a un conjunto específico de funciones y clases programadas en el archivo `Utilerias.py`, las cuales encapsulan tanto la representación de las partículas como los procesos de evaluación y movimiento del algoritmo PSO. El algoritmo sigue directamente las cinco fases descritas en la justificación del algoritmo heurístico, utilizando funciones y clases desarrolladas específicamente para representar el modelo de optimización y automatizar cada etapa del proceso.

Parametrización del modelo

En esta primera fase se establecieron las constantes que determinan tanto el comportamiento de la función fitness como la dinámica del algoritmo PSO. Se definió una configuración de 25 sensores por partícula, junto con los pesos asociados a las variables ambientales y agronómicas: temperatura (0.3), salinidad (-0.5) y cultivo (0.4). También se asignaron ponderaciones particulares a los tipos de cultivo considerados: maíz (1.0), chile (1.5) y tomate (0.8). Estas constantes permiten ajustar la influencia relativa de cada variable en la evaluación de las soluciones propuestas.

Para el algoritmo de búsqueda se configuró un enjambre de 10 partículas y un total de 100 iteraciones, con un factor de inercia de 0.7 y coeficientes cognitivo y social de 1.5. El conjunto de datos se cargó desde el archivo `Dataset_Enjambre.xlsx` mediante la función `cargar_dataset()`, la cual devuelve un `DataFrame` de *pandas* con los 100 puntos geográficos y sus variables asociadas.

```

proyectosModuloIII > OptimizacionSensoresDeRiegoEnjambre > Main_Enjambre.py > ...
2
3
4 def main():
5     # Cargar el dataset e imprimirlo en pantalla
6     dataset = cargar_dataset(verbose=True)
7
8     # Ejemplo de uso posterior
9     print("\nCantidad de registros:", len(dataset))
10    print("Columnas disponibles:", list(dataset.columns))
11
12

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

	Humedad (%)	Cultivo	Elevación (m)	Salinidad (ds/m)	Temperatura (°C)	Latitud	Longitud
0	35.4	Maíz	32.1	1.18	30.9	25.525269	-108.480884
1	20.0	Maíz	40.8	3.30	27.5	25.600618	-108.519701
2	43.3	Maíz	21.9	1.17	31.8	25.577411	-108.465669
3	36.3	Maíz	41.7	2.93	25.3	25.616337	-108.470516
4	17.3	Chile	31.8	1.91	29.1	25.522286	-108.480179
..	...	...	...	...	...	...	...
95	29.7	Tomate	14.5	1.54	28.3	25.567005	-108.462403
96	28.9	Maíz	39.9	1.64	31.4	25.533387	-108.472275
97	25.5	Maíz	37.8	0.87	20.8	25.555069	-108.457875
98	17.5	Chile	23.3	1.58	22.3	25.569962	-108.457690
99	12.5	Tomate	43.7	1.65	23.3	25.570974	-108.420756

[100 rows x 7 columns]

Cantidad de registros: 100
Columnas disponibles: ['Humedad (%)', 'Cultivo', 'Elevación (m)', 'Salinidad (ds/m)', 'Temperatura (°C)', 'Latitud', 'Longitud']

Img.1 Resultado de la carga correcta del Data set

Con el Data set ya disponible, se inicializó el objeto *FuncionFitness*, responsable de estimar las variables ambientales y calcular el valor fitness de cada partícula. También se creó el objeto *MovimientoParticulas*, encargado de aplicar la ecuación de movimiento del PSO y de ordenar internamente las coordenadas de cada sensor antes de efectuar el desplazamiento.

Generación del enjambre inicial

La generación del enjambre se implementó mediante la función *generar_enjambre(n_particulas, n_sensores, dataset)*. Esta función obtiene los límites mínimos y máximos de latitud y longitud presentes en el Data set y utiliza dichos rangos para crear posiciones iniciales aleatorias dentro del área geográfica de estudio.

```
4 def main():
5     n_particulas = 3
6     n_sensores = 5
7
8     dataset = cargar_dataset(verbose=False)
9
10    enjambre = generar_enjambre( n_particulas=n_particulas, n_sensores=n_sensores, dataset=dataset,
11                                verbose=True
12    )
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
=== ENJAMBRE GENERADO ===

Particula 1
x,y
25.536645, -108.451460
25.552244, -108.456698
25.547622, -108.449458
25.617302, -108.479504
25.550280, -108.441931

Particula 2
x,y
25.593637, -108.497201
25.540426, -108.490533
25.560439, -108.423035
25.606741, -108.428042
25.541115, -108.454453

Particula 3
x,y
25.539371, -108.429457
25.615271, -108.463632
25.546864, -108.481355
25.556871, -108.492896
25.592234, -108.495575

Número de partículas generadas: 3
```

Img.2 Generación aleatoria de coordenadas de sensores dentro del espacio del proyecto

Cada partícula se representa como una estructura compuesta por tres elementos: posición actual, vector de velocidades y óptimo local. La posición actual contiene el vector de coordenadas y un valor fitness inicial igual a cero; el vector de velocidades almacena pequeños desplazamientos aleatorios para cada coordenada; y el óptimo local se inicializa como una copia exacta de la posición actual. El resultado es un enjambre de soluciones distribuidas aleatoriamente en el espacio de búsqueda, listo para su evaluación.

Evaluación de partículas

La evaluación del enjambre se realiza con la función *evaluar_enjambre(enjambre, fitness)*, la cual recorre cada partícula y utiliza *FuncionFitness.evaluar_particula()* para calcular su valor fitness total.

Dentro de este proceso, *evaluar_particula()* analiza cada par de coordenadas de la partícula y llama a *estimar_sensor(sx, sy)*, función que estima los valores de humedad, temperatura y salinidad correspondientes a la ubicación propuesta del sensor. A partir de estos valores se calcula un fitness individual y se acumula la suma total de la partícula. El método devuelve tanto el fitness global como un diccionario con los promedios ambientales de todos los sensores.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

=== ENJAMBRE GENERADO ===

Partícula 1
x,y
25.530750, -108.492732
25.583181, -108.511107

Partícula 2
x,y
25.526022, -108.461724
25.538871, -108.501833

=====
EVALUANDO PARTÍCULAS
=====

##### PARTÍCULA 1 #####

=== EVALUACIÓN DE PARTÍCULA ===

Sensor 1
Posición: (25.530750, -108.492732)
Humedad: 24.0074
Temperatura: 29.8405
Salinidad: 2.1587
Fitness sensor: 8.3188

Sensor 2
Posición: (25.583181, -108.511107)
Humedad: 24.3098
Temperatura: 29.6487
Salinidad: 2.2602
Fitness sensor: 8.2105

PROMEDIOS DE PARTÍCULA
Humedad promedio: 24.1586
Temperatura promedio: 29.7446
Salinidad promedio: 2.2094

FITNESS TOTAL: 16.5293
```

Img.3 Evaluación de sensores dentro de una partícula, el resultado del fitness de la partícula es la sumatoria del fitness de los sensores que contiene.

Con el fitness calculado, *evaluar_enjambre()* actualiza el valor almacenado en la posición actual de cada partícula y compara el resultado con su óptimo local. Si la nueva posición presenta un mejor desempeño, el óptimo local es reemplazado. Al concluir el recorrido del enjambre, la función devuelve la mejor partícula de la iteración en la forma [*vector_variables*, *valor_fitness*], denominada óptimo del enjambre.

```

Enjambre/Main_Enjambre.py

=== ENJAMBRE GENERADO ===

Partícula 1
x,y
25.550077, -108.440517
25.538903, -108.508464

Partícula 2
x,y
25.605451, -108.424592
25.618295, -108.426978

Partícula 3
x,y
25.549892, -108.426895
25.604566, -108.450130

Partícula 4
x,y
25.575760, -108.486273
25.601688, -108.512805

Partícula 5
x,y
25.555022, -108.453553
25.535956, -108.486370

=====
EVALUANDO ENJAMBRE
=====

Partícula 1
Humedad promedio: 23.4139
Temperatura promedio: 30.0248
Salinidad promedio: 2.0330
Fitness: 16.8739

Partícula 2
Humedad promedio: 23.4139
Temperatura promedio: 30.0248
Salinidad promedio: 2.0330
Fitness: 16.8739

Partícula 3
Humedad promedio: 23.4139
Temperatura promedio: 30.0248
Salinidad promedio: 2.0330
Fitness: 16.8739

Partícula 4
Humedad promedio: 23.4139
Temperatura promedio: 30.0248
Salinidad promedio: 2.0330
Fitness: 16.8739

Partícula 5
Humedad promedio: 23.4139
Temperatura promedio: 30.0248
Salinidad promedio: 2.0330
Fitness: 16.8739

=====
MEJOR PARTÍCULA DEL ENJAMBRE
=====
Número de partícula: 4
Fitness: 16.8750

=====
ÓPTIMO DEL ENJAMBRE
=====
Fitness máximo: 16.8750

```

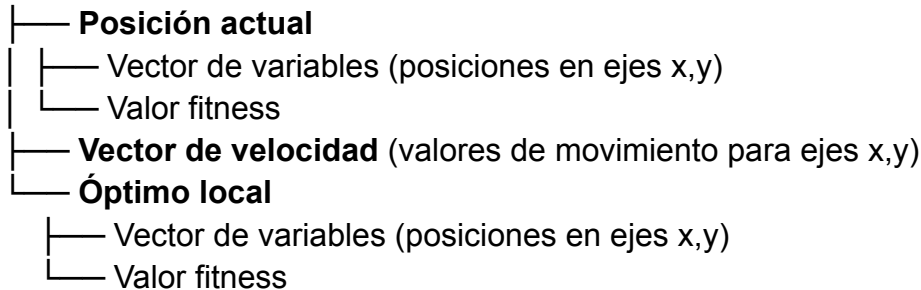
Img.4, 5 Evaluación de enjambre, resultado de múltiples evaluaciones de partículas

Evolución del enjambre

La fase evolutiva se implementa mediante *MovimientoParticulas.movimiento(particula, optimo_social)*. Este método recibe una partícula y el óptimo social y actualiza sus posiciones y velocidades de acuerdo con la ecuación clásica del PSO. Pero antes de aplicar la ecuación, la partícula es reorganizada con *ordenar_particula_completa()*, que a su vez utiliza *calcular_prioridad()*, que establece una correlación consistente entre los sensores de la partícula y los sensores del óptimo social mediante distancias euclidianas y un criterio de prioridad basado en cercanía relativa. Esta etapa garantiza que cada sensor sea atraído hacia el punto socialmente equivalente y no hacia una coordenada arbitraria.

La estructura de datos de una partícula se puede representar de la siguiente forma:

Partícula



Es por esto que re ordenar los sensores requiere tanto de un orden del vector de posición actual, como del vector de velocidad y del vector de posiciones en el óptimo local.

```
=====
ORDENANDO PARTÍCULA
=====

=====
ÓPTIMO SOCIAL
=====
Índice   X           Y
0        25.585760   -108.518146
1        25.584483   -108.495400
2        25.577487   -108.490882
3        25.555152   -108.472920
4        25.562526   -108.512478

=====
ORDENAMIENTO DE POSICIÓN ACTUAL
=====
Índice   X           Y           VX           VY           Índice Social   Distancia
0        25.589234   -108.422657   -0.003163   -0.006803   0              0.095552
1        25.572006   -108.459170   -0.007595   -0.005987   3              0.021751
2        25.555615   -108.489516   -0.003679   -0.006691   4              0.023980
3        25.581157   -108.486297   0.003997    0.001284    2              0.005872
4        25.617872   -108.482341   0.006844    -0.005464   1              0.035852

=====
ORDENAMIENTO DE ÓPTIMO LOCAL
=====
Índice   X           Y           Índice Social   Distancia
0        25.589234   -108.422657   0              0.095552
1        25.572006   -108.459170   3              0.021751
2        25.555615   -108.489516   4              0.023980
3        25.581157   -108.486297   2              0.005872
4        25.617872   -108.482341   1              0.035852

=====
PARTÍCULA ORDENADA
=====
[[[np.float64(25.58923444295658), np.float64(-108.42265696971958), np.float64(25.61787208697436), np.float64(-108.48234087258965), np.float64(25.581156746659637), np.float64(-108.4862971431256), np.float64(25.57200645983561), np.float64(-108.45917011302613), np.float64(25.55561480853754), np.float64(-108.48951612950381)], 40.71262258799748], [np.float64(-0.0031628289520093697), np.float64(-0.0068028428969320424), np.float64(0
```

Img.6 Impresión del proceso de ordenamiento: Se asignan índices a los sensores del óptimo social y local y la posición actual calculan sus sensores más cercanos y adquieren un índice ajeno que les otorgará el siguiente orden.

Una vez ordenados los vectores, el método *movimiento()* genera valores aleatorios y calcula las nuevas velocidades considerando el término de inercia, el término

cognitivo y el término social. Las velocidades obtenidas se suman a las coordenadas actuales para producir nuevas posiciones. El resultado es una partícula completa con su posición actual, velocidades actualizadas y óptimo local intacto.

=====						
PARTÍCULA ORIGINAL						
=====						
Sensor	PX	PY	LBEST_X	LBEST_Y	VX	VY
0	25.581464	-108.462262	25.581464	-108.462262	0.002475	0.004790
1	25.586804	-108.464410	25.586804	-108.464410	-0.004986	-0.007062
2	25.578725	-108.485748	25.578725	-108.485748	0.004721	0.007749
3	25.530736	-108.516805	25.530736	-108.516805	0.001923	0.001467
4	25.566193	-108.455432	25.566193	-108.455432	-0.006136	0.000834
=====						
PARTÍCULA ORDENADA						
=====						
Sensor	PX	PY	LBEST_X	LBEST_Y	VX	VY
0	25.566193	-108.455432	25.566193	-108.455432	-0.006136	0.000834
1	25.581464	-108.462262	25.581464	-108.462262	0.002475	0.004790
2	25.578725	-108.485748	25.578725	-108.485748	0.004721	0.007749
3	25.530736	-108.516805	25.530736	-108.516805	0.001923	0.001467
4	25.586804	-108.464410	25.586804	-108.464410	-0.004986	-0.007062
=====						
APLICACIÓN DE MOVIMIENTO						
=====						
Var	X(t)	w*v	c1*r1*(pbest-x)	c2*r2*(gbest-x)	V(t+1)	X(t+1)
S1-X	25.566193	-0.004295	0.000000	-0.000607	-0.004902	25.561292
S1-Y	-108.455432	0.000584	0.000000	-0.003696	-0.003112	-108.458545
S2-X	25.581464	0.001733	0.000000	-0.038786	-0.037054	25.544410
S2-Y	-108.462262	0.003353	0.000000	-0.006882	-0.003529	-108.465790
S3-X	25.578725	0.003305	0.000000	0.000617	0.003922	25.582648
S3-Y	-108.485748	0.005424	0.000000	-0.013040	-0.007616	-108.493364
S4-X	25.530736	0.001346	0.000000	0.002710	0.004056	25.534793
S4-Y	-108.516805	0.001027	0.000000	0.007828	0.008855	-108.507950
S5-X	25.586804	-0.003490	0.000000	-0.011909	-0.015399	25.571405
S5-Y	-108.464410	-0.004943	0.000000	-0.001037	-0.005980	-108.470390

Img.7 Tablas con los procesos de transformación de los valores de una partícula al ser pasada por la función de movimiento

Durante el programa principal, este procedimiento se aplica a todas las partículas del enjambre en cada iteración. Después de cada actualización, el enjambre se evalúa nuevamente con *evaluar_enjambre()*, se registra el fitness del óptimo del enjambre y se compara con el óptimo social. Si la mejor partícula de la iteración supera el mejor resultado histórico, se actualiza el óptimo social del algoritmo.

Análisis y presentación de resultados

Al concluir las iteraciones, la mejor solución encontrada queda almacenada en *optimo_social*. Esta partícula se evalúa nuevamente con

FuncionFitness.evaluar_particula() para recuperar tanto su fitness total como los promedios globales de humedad, temperatura y salinidad.

Con esta información se imprime una tabla que muestra, para cada sensor, sus coordenadas geográficas, los valores ambientales estimados y su fitness individual. También se presentan los promedios de la partícula, la sumatoria de los fitness individuales y el fitness total del óptimo global.

El programa calcula además el porcentaje de crecimiento o decrecimiento respecto a la mejor partícula obtenida en la inicialización, lo que permite cuantificar la mejora lograda por el algoritmo. El historial de fitness almacenado durante la ejecución se representa gráficamente mediante *matplotlib*, generando una curva que muestra la evolución del rendimiento del enjambre y el proceso de convergencia hacia la solución final.

6.5 Evaluación del rendimiento de la solución

Las iteraciones del enjambre guardan en un historial la mejor partícula por iteración, lo que ayuda a generar una gráfica de la convergencia en la optimización. El algoritmo se probó con diversas configuraciones en los parámetros, para buscar una optimización en el resultado del valor fitness, y se pudieron obtener conclusiones interesantes.

```
ITERACIÓN 38 - Mejor fitness del enjambre: 44.414452
ITERACIÓN 39 - Mejor fitness del enjambre: 43.943903
ITERACIÓN 40 - Mejor fitness del enjambre: 44.586107
Nuevo óptimo social encontrado
ITERACIÓN 41 - Mejor fitness del enjambre: 44.556797
ITERACIÓN 42 - Mejor fitness del enjambre: 44.555936
ITERACIÓN 43 - Mejor fitness del enjambre: 44.782743
Nuevo óptimo social encontrado
ITERACIÓN 44 - Mejor fitness del enjambre: 44.424039
ITERACIÓN 45 - Mejor fitness del enjambre: 44.538167
ITERACIÓN 46 - Mejor fitness del enjambre: 44.751845
ITERACIÓN 47 - Mejor fitness del enjambre: 45.123894
Nuevo óptimo social encontrado
ITERACIÓN 48 - Mejor fitness del enjambre: 45.154515
Nuevo óptimo social encontrado
ITERACIÓN 49 - Mejor fitness del enjambre: 44.754635
ITERACIÓN 50 - Mejor fitness del enjambre: 44.856934
ITERACIÓN 51 - Mejor fitness del enjambre: 45.505531
Nuevo óptimo social encontrado
ITERACIÓN 52 - Mejor fitness del enjambre: 45.073208
ITERACIÓN 53 - Mejor fitness del enjambre: 45.287262
ITERACIÓN 54 - Mejor fitness del enjambre: 44.939501
ITERACIÓN 55 - Mejor fitness del enjambre: 45.249739
ITERACIÓN 56 - Mejor fitness del enjambre: 44.979186
ITERACIÓN 57 - Mejor fitness del enjambre: 45.001962
ITERACIÓN 58 - Mejor fitness del enjambre: 45.155605
ITERACIÓN 59 - Mejor fitness del enjambre: 44.836440
ITERACIÓN 60 - Mejor fitness del enjambre: 45.065748
ITERACIÓN 61 - Mejor fitness del enjambre: 45.041533
ITERACIÓN 62 - Mejor fitness del enjambre: 45.194220
ITERACIÓN 63 - Mejor fitness del enjambre: 44.821069
ITERACIÓN 64 - Mejor fitness del enjambre: 45.452688
ITERACIÓN 65 - Mejor fitness del enjambre: 45.427508
ITERACIÓN 66 - Mejor fitness del enjambre: 45.244842
ITERACIÓN 67 - Mejor fitness del enjambre: 45.499999
ITERACIÓN 68 - Mejor fitness del enjambre: 45.051768
ITERACIÓN 69 - Mejor fitness del enjambre: 45.161640
ITERACIÓN 70 - Mejor fitness del enjambre: 45.653337
Nuevo óptimo social encontrado
ITERACIÓN 71 - Mejor fitness del enjambre: 45.459076
ITERACIÓN 72 - Mejor fitness del enjambre: 45.596277
```

Img.8 Ejemplo de ejecución del algoritmo y su convergencia a un óptimo. El parámetro con más influencia en el algoritmo más que los pesos es el de la cantidad de sensores por partícula, ya que añade dimensionalidad al algoritmo y hace crecer exponencialmente su costo de computación. El algoritmo se comparó con diversos valores en el parámetro de cantidad de sensores, y se obtuvieron claros resultados.

```

ITERACIÓN 97 - Mejor fitness del enjambre: 46.452577
ITERACIÓN 98 - Mejor fitness del enjambre: 46.578651
ITERACIÓN 99 - Mejor fitness del enjambre: 46.766058
ITERACIÓN 100 - Mejor fitness del enjambre: 46.420872

=====
RESULTADO FINAL
=====

Fitness óptimo global: 46.780591

=====
SENSORES GANADORES
=====

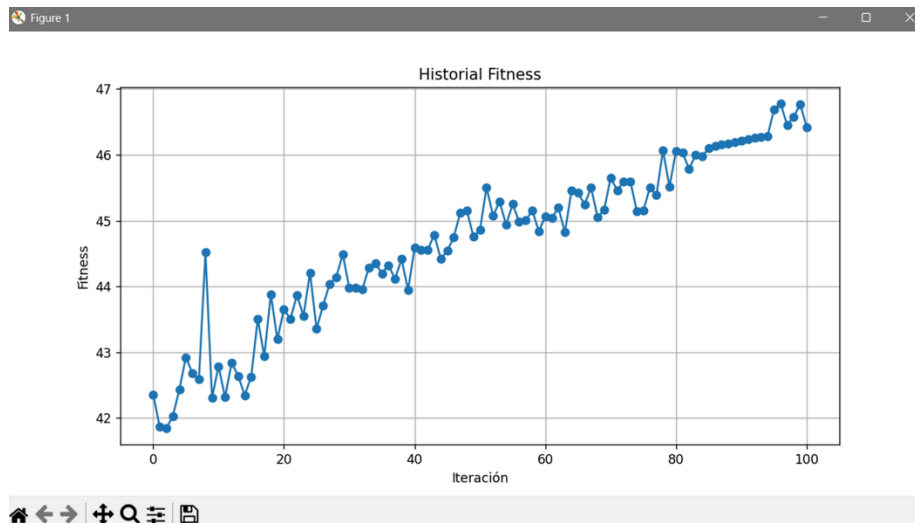
```

Sensor	X	Y	Humedad	Temp	Salinidad	Fitness
1	25.537450	-108.498197	25.2857	35.3283	2.4878	9.8006
2	25.571079	-108.506080	24.3396	30.4843	2.0359	8.5733
3	25.594061	-108.486584	25.6792	29.2733	2.0597	8.1981
4	25.569563	-108.500888	23.2558	32.6014	1.8475	9.3026
5	25.571940	-108.492518	16.7274	36.1093	0.7458	10.9059

```

=====
PROMEDIOS FINALES
=====
Humedad promedio: 23.0576
Temperatura promedio: 32.7593
Salinidad promedio: 1.8353
Fitness total: 46.7806
Sumatoria fitness: 46.7806
Cambio porcentual respecto al inicio: 10.45%

```



Img.9, 10 - Resultados del algoritmo con 5 sensores por partícula

```

=====
RESULTADO FINAL
=====

Fitness óptimo global: 84.979908

=====
SENSORES GANADORES
=====

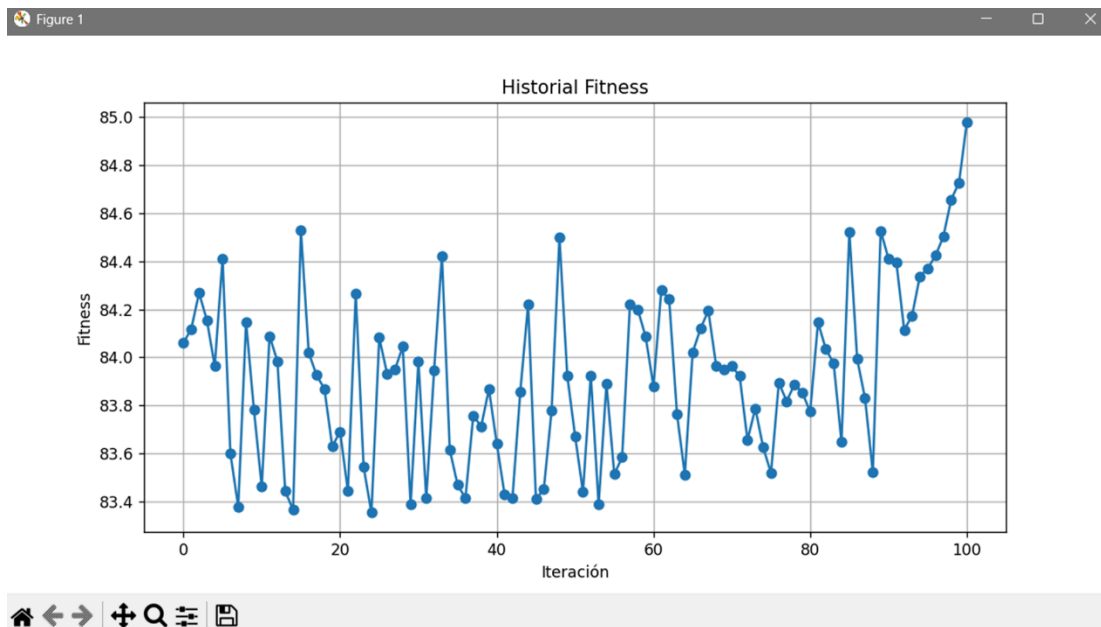
```

Sensor	X	Y	Humedad	Temp	Salinidad	Fitness
1	25.535732	-108.534721	24.8199	29.5916	1.9783	8.3344
2	25.521240	-108.467182	24.1913	29.7131	2.0956	8.3121
3	25.618997	-108.559707	25.1353	29.6283	2.0778	8.2956
4	25.601257	-108.507813	15.9099	31.1997	2.0187	8.7966
5	25.535599	-108.471202	25.1888	30.3793	2.1605	8.4795
6	25.543029	-108.482079	24.2259	30.0140	2.1432	8.3786
7	25.551227	-108.445094	18.4925	32.6965	1.8227	9.3436
8	25.577118	-108.505203	23.6298	30.8649	2.2069	8.6020
9	25.545561	-108.457905	23.9255	29.5483	1.9996	8.3107
10	25.679844	-108.441414	25.7973	29.1124	2.1057	8.1269

```

=====
PROMEDIOS FINALES
=====
Humedad promedio: 23.1316
Temperatura promedio: 30.2748
Salinidad promedio: 2.0609
Fitness total: 84.9799
Sumatoria fitness: 84.9799
Cambio porcentual respecto al inicio: 1.09%

```



Img.11, 12 - Resultados del algoritmo con 10 sensores por partícula

Fitness óptimo global: 210.141760

=====

SENSORES GANADORES

=====

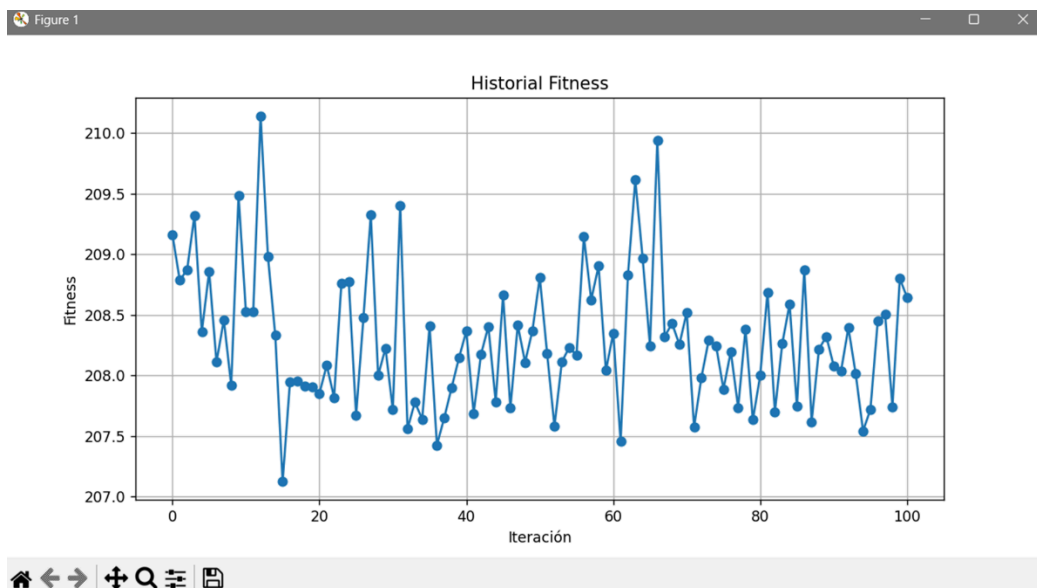
Sensor	X	Y	Humedad	Temp	Salinidad	Fitness
1	25.546302	-108.488807	23.4141	30.2783	2.1892	8.4349
2	25.595173	-108.421327	25.9257	28.6626	2.0588	8.0154
3	25.528739	-108.459240	24.7110	29.9003	2.0389	8.3966
4	25.567812	-108.671127	25.1096	29.5498	2.0621	8.2799
5	25.525989	-108.428595	25.6981	30.2748	2.0099	8.5235
6	25.545233	-108.463886	24.4579	29.2891	2.1361	8.1647
7	25.466667	-108.482976	24.7555	29.6386	2.0599	8.3076
8	25.507059	-108.445942	24.7319	29.8031	2.0509	8.3615
9	25.540762	-108.438601	25.2290	31.2854	2.2252	8.7190
10	25.600380	-108.496138	39.3680	34.0071	1.4659	9.9152
11	25.535199	-108.495895	24.0186	30.7366	2.1753	8.5793
12	25.528024	-108.453118	24.6637	29.9851	2.0127	8.4352
13	25.682078	-108.425785	25.7810	29.0964	2.1003	8.1248
14	25.571411	-108.493804	22.9457	31.4981	1.7722	9.0093
15	25.520822	-108.484575	24.0438	29.6252	2.0819	8.2926
16	25.565021	-108.231432	25.2170	29.3739	2.0685	8.2239
17	25.509746	-108.501354	24.5895	29.5734	2.0762	8.2799
18	25.538024	-108.462981	25.2508	29.9869	2.0706	8.4068
19	25.525560	-108.428546	25.6566	30.2541	2.0113	8.5166
20	25.614851	-108.505644	25.0270	30.1259	2.1113	8.4281
21	25.761866	-108.478820	25.4489	29.3243	2.0884	8.1991
22	25.588799	-108.394102	25.3958	29.0202	2.0630	8.1206
23	25.612326	-108.437207	27.0203	28.3425	2.1340	7.8817
24	25.558829	-108.470665	24.5089	29.1489	2.0515	8.1650
25	25.578120	-108.526982	25.3916	29.7593	2.0263	8.3607

=====

PROMEDIOS FINALES

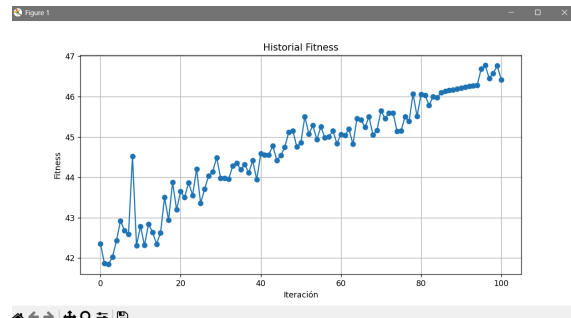
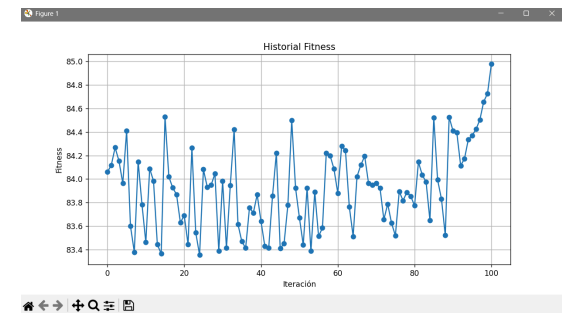
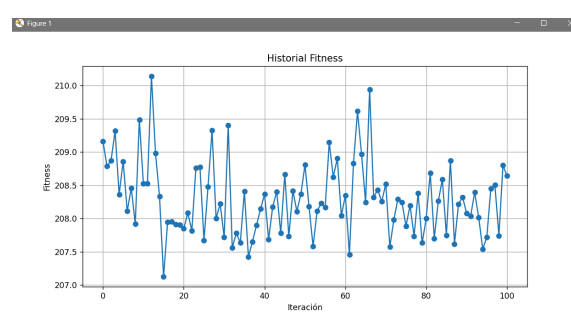
=====

Humedad promedio: 25.5344
Temperatura promedio: 29.9416
Salinidad promedio: 2.0456
Fitness total: 210.1418
Sumatoria fitness: 210.1418
Cambio porcentual respecto al inicio: 0.47%



Img.13, 14 - Resultados del algoritmo con 25 sensores por partícula

Transportando esto a una tabla de resultados:

Num. Sensores	% Mejora del algoritmo	Valor Fitness	Gráfica de mejora
5	10.45%	46.78	
10	1.09%	84.97	
25	0.47%	210.14	

Los resultados obtenidos muestran una relación clara entre la cantidad de sensores utilizados y el desempeño del algoritmo de optimización. Cuando se emplearon 5 sensores, el algoritmo logró una mejora del 10.45% respecto a la mejor solución inicial, alcanzando un valor fitness de 46.78. Con 10 sensores, la mejora se redujo a 1.09%, aunque el fitness total aumentó considerablemente hasta 84.97. Finalmente, al utilizar 25 sensores, el porcentaje de mejora fue de apenas 0.47%, mientras que el valor fitness alcanzó 210.14. Esta tendencia indica

que, a medida que aumenta el número de sensores, el espacio de búsqueda se vuelve significativamente más complejo y el algoritmo presenta una menor capacidad de mejora relativa dentro del mismo número de iteraciones, lo que refleja una disminución en su eficiencia de convergencia. Sin embargo, el incremento sostenido del valor fitness demuestra que una mayor cantidad de sensores permite representar con mayor detalle las condiciones del terreno y obtener estimaciones más precisas y robustas de la humedad y de las variables ambientales asociadas. En términos prácticos, esto evidencia un compromiso entre eficiencia computacional y precisión del modelo: configuraciones con pocos sensores permiten mejoras más rápidas, mientras que configuraciones con un mayor número de sensores ofrecen soluciones de mayor calidad y capacidad descriptiva del sistema agrícola.

8. Conclusiones

El desarrollo del presente proyecto permitió demostrar que el algoritmo de Optimización por Enjambre de Partículas PSO constituye una herramienta adecuada para abordar el problema de ubicación de sensores de humedad en sistemas agrícolas. La representación de cada partícula como un conjunto de coordenadas geográficas, junto con una función fitness basada en variables como humedad, temperatura, salinidad y tipo de cultivo, permitió modelar y medir de manera efectiva la interacción entre las características del terreno y la calidad de una configuración de sensores. Uno de los principales retos fue la incorporación de un mecanismo de ordenamiento previo al movimiento de las partículas garantizó la coherencia espacial en el movimiento entre sensores equivalentes, permitiendo que las fuerzas cognitivas y sociales del algoritmo actuarán de forma consistente durante el proceso de búsqueda.

Los resultados obtenidos evidenciaron que el algoritmo es capaz de mejorar progresivamente las configuraciones iniciales y converger hacia soluciones de alta calidad, además de ser personalizable a las prioridades y los recursos de distintos casos. Se observó que, conforme aumenta la cantidad de sensores, el espacio de búsqueda se vuelve considerablemente más complejo y el porcentaje de mejora respecto a la solución inicial disminuye. No obstante, el valor fitness total crece de manera significativa, lo que indica que un mayor número de sensores proporciona una representación más detallada y precisa de las condiciones ambientales del terreno. Este comportamiento confirma la existencia de un compromiso entre eficiencia computacional y precisión del modelo, en donde configuraciones pequeñas permiten una convergencia más rápida, mientras que configuraciones

más extensas ofrecen soluciones con mayor capacidad descriptiva y potencial para apoyar decisiones de riego más precisas.

9. Referencias

[1] Implementación de algoritmo de índice ndvi en la detección de estrés hídrico en cultivos (Junio 2023)

https://github.com/ramilcar525/Automatic-FlightPlan_Documentacion